

foop

Stop writing scripts for throwaway on-chain workloads.
Just Foop it.

Define → **Simulate** → **Review** → **Execute** → **Observe**

*Define, simulate, and run repeatable on-chain workloads: no one-off scripts. **Describe them in plain English; run multi-step flows safely.***

The throwaway-script tax

TODAY

Explorer → click → sign → wait

repeat x100

...and for a multi-step flow (wrap, approve, supply, borrow) you hand-roll a Foundry / viem script, get the ordering wrong, and burn testnet gas debugging it.

Empty testnets

Fresh deployments have no activity to build against.

Starved indexers

Subgraphs and pipelines have no events to index.

Empty screenshots

Staging frontends render blank tables and zero balances.

Fragile step ordering

Dependent calls (approve → supply → borrow) break silently.

One declarative flow, four safe stages

1 Define

Chain, contract, function, args, value, repeat.

no signing

2 Simulate

Check balances, estimate gas, decode reverts, count what's executable.

no signing

3 Execute

Approve once, then submit in order with nonce management + retries.

signs

4 Observe

Confirmed / failed / skipped, gas totals, per-tx explorer links.

no signing

Why it's safe

Simulation is free & repeatable · never signs until you approve · iteration count is a hard upper bound · never custodies your keys.

Two execution modes

Single-step: simulate once, then execute. **Multi-step: staged** — each step re-checked against live chain state right before it signs.

Just say what you want

foop ai



```
$ foop ai "wrap 0.01 ETH 10 times  
on Base Sepolia"
```

PROPOSED WORKLOAD

```
Chain  Base Sepolia (84532)  
Step 1  deposit()  x10  → 0x4200...0006  
value: 0.01 ETH
```

WHAT YOU'RE ABOUT TO SIGN

- Each tx calls deposit() — wraps ETH → WETH
- All 10 executable • total value 0.1 ETH
- Est. fee ~0.0000027 ETH

```
Execute 10 transaction(s) now? > No / Yes
```

1

Understands the request

Maps the chain name to its id and pulls out contract, amount, and repeat count.

2

Resolves the real function

Looks up the contract's ABI and picks the matching writable function — no guessed signatures.

3

Asks when unsure

Prompts for anything it can't infer. It never invents addresses or token amounts.

4

Simulates, explains, executes

Grounds a plain-English "what you'll sign" in real numbers, then runs the guarded pipeline — confirm defaults to No.

Real dependent flows, run in order



```
$ foop run workload.json
```

Staged execution (multi-step workload)

Step 1/4	deposit()	wrap 0.1 ETH	✓
Step 2/4	approve()	WETH → pool	✓
Step 3/4	supply()	0.01 WETH ×10	✓
Step 4/4	borrow()	10 USDC ×10	✓

↳ re-checked live: collateral now exists

22 confirmed 0 failed 0 skipped

► Re-simulated per step, live

Each step is checked against fresh chain state right before it signs: so borrow is only tried once supply has landed.

► Dependency-aware halting

A blocked or failed step halts the steps that depend on it; a funds-capped step runs what it can afford and continues.

► Honest, up-front approval

One confirmation covers the run. Earlier steps sign for real before later ones can be verified: foop tells you so.

Also under the hood: multi-source ABI resolution · actionable revert diagnostics · version-controlled JSON workloads · foop ai · full CLI (init · save · list · history · templates)

Describe the work once, simulate it safely, run it repeatably.